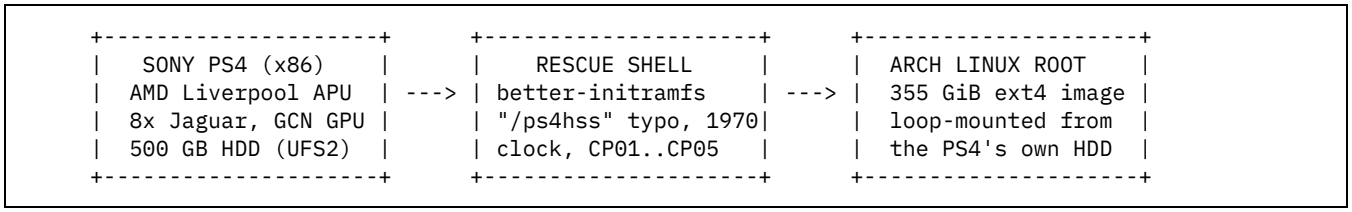
JUPI Field Manual

A Sony PS4 that fell into a rescue shell and came out as an Arch Linux instrument.
How it is built, how it boots, how its memory behaves, and how to bring it back.



Host / user	`playstation` / `pi`
Kernel	`6.15.4-crashnt-4.7_no_built-in-fw` (x86_64, custom PS4 build)
Root	`/dev/loop0` ext4 `JUPIROOT` 355 GiB, UUID `70dbf870-bcb6-4be9-9fb5-f25c4718b71a`
Source of truth	`JUPI-ASBUILT-SCAN-20261110T1610Z.txt` — 279,105 bytes sha256 `3559b9342a67edecf5f9adcfbc196f875b53b1e989f1552fab9eb7e496053c8d`
Edition	PUBLIC EDITION — LAN/tailnet addresses, key paths and full fingerprints redacted
Compiled	Tue Nov 10 2026, 8:43 AM PST, by Grok Bot for Kam (Kamron Noah Dabe)

How to read this manual

Times. Every time in this manual is Pacific Standard Time (PST, UTC−8) unless it carries its own zone. JUPI's own clock is set to `Europe/London` (inherited from the 2022 image), which in November equals UTC. So a JUPI journal line stamped `15:47 GMT` happened at **7:47 AM PST**.

Evidence words. *Observed* means read directly from the scan or a receipt. *Inferred* means reasoned from observed facts and labeled as such. **UNKNOWN** means the evidence does not establish it — no guessing has been used to fill those gaps.

Secrets. No private key, credential, unlock key or pairing file appears here. SSH keys are identified by fingerprint and comment only. This public edition replaces LAN/tailnet addresses and key-file paths with role placeholders such as `<jupi-lan-ip>`, and truncates fingerprints.

Where to start. In trouble right now? Go straight to **Section 7, Freeze & Recovery Runbook**. New to the box? Read Section 1, then Section 3.

§	Section	What you get
1	What JUPI is	One-page summary, layer diagram, key facts
2	Origin: the rescue-shell story	How a typo created the platform
3	Boot chain	Step diagram of the root selector, exit codes, fallbacks
4	Storage layout	Disks, partitions, the image and its headroom
5	As-built OS configuration	sysctl, systemd, zram, earlyoom, display, firewall
6	Unified memory model	Why a desktop app can starve the machine
7	**Freeze & recovery runbook**	Exact commands, in order, with expected output
8	Lanes & access map	Every way in and out, with key fingerprints
9	Life timeline	Nov 5 → Nov 10, from receipts and the journal
10	Known open issues	What is wrong or unproven right now
11	Receipts appendix	Paths, sizes, sha256, fingerprints, boot list

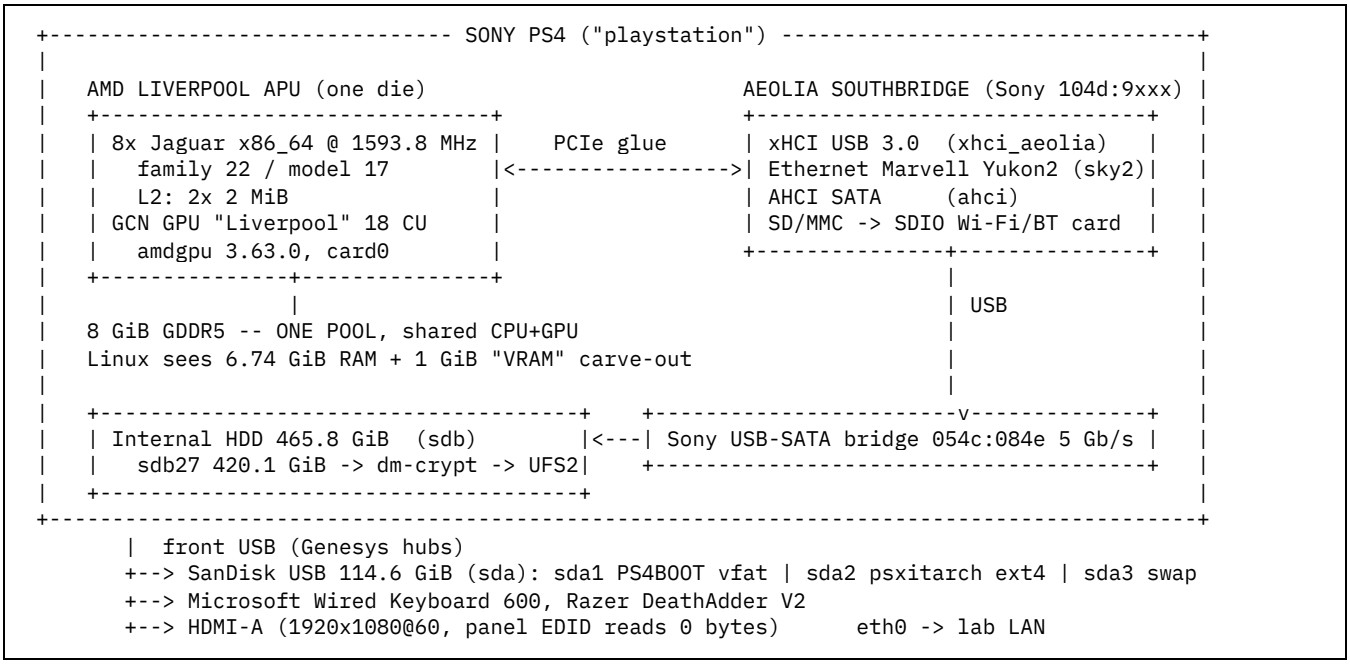
1. What JUPI is

JUPI (Joint Unix Platform Integrator) is a Sony PlayStation 4 running Arch Linux as a lab instrument. It is x86_64 — AMD Jaguar cores, **not ARM** — even though its user is called `pi` and its locale is `en-GB`/London. Those are leftovers from the 2022 community *psxitarch* PS4 Arch image it grew out of, not signs of a Raspberry Pi.

Its root filesystem is **not on the USB stick**. Root is a 355 GiB ext4 *image file* stored on the PS4's own internal 500 GB hard drive. The drive sits behind Sony's internal USB-SATA bridge and is encrypted. Each boot, the initramfs decrypts the drive, mounts Sony's UFS2 filesystem at `/ps4hdd`, attaches the image as `/dev/loop0` and boots it. The SanDisk stick is kept only as the fallback root and as a boot medium.

LAYER	WHAT	DETAIL	WHERE
7	Workloads	jupi-agent, ollama, science tools, XFCE desktop, ChatGPT, Grok Bot	/ (loop0)
6	Userspace	Arch Linux (rolling), systemd, lightdm autologin pi -> XFCE	/ (loop0)
5	Root filesystem	ext4 "JUPIROOT", 355 GiB	/dev/loop0
4	Image file	jupi-linux-2a79fafeb62d61ff.img	/ps4hdd/home/
3	Backing filesystem	Sony UFS2, 414 GiB, mounted RW	/ps4hdd
2	Disk decryption	dm-crypt plain, aes-xts-plain64, 256-bit, map "jupi-boot-internal"	/dev/mapper/ jupi-boot-internal
1	Physical partition	sdb27, 420.1 GiB (GPT, 15 parts)	internal HDD, 465.8 GiB
0	Kernel + initramfs	6.15.4-crashnt-4.7 + root selector	loaded by PS4 payload (USB or HDD copy: UNKNOWN)
-1	Console OS + homebrew loader	Orbis OS -> linux-1024mb.bin	PS4 firmware/HDD

Hardware block diagram



Why the HDD is "USB". Linux sees the internal drive as `sdb` with model `USB-SATA Bridge` (Sony 054c:084e) on the Aeolia xHCI bus. Removing USB support would remove the internal drive too.

Key facts

Item	Value	Status
CPU	8× AMD Jaguar (`DG1000FGF84HT`), x86_64, 1593.786 MHz TSC, 1 thread/core	observed
GPU	Liverpool GCN, 18 active CUs, `amdgpu`, VRAM 1 GiB, GTT cap 3,618,242,560 B (3.37 GiB)	observed
RAM	MemTotal 7,070,228 kB (6,904 MiB in `free -m`); unified with GPU	observed
Swap	zram0 2 GiB lz4 prio 100 + zram1 2 GiB zstd prio 200 = 4 GiB, RAM-backed	observed
Kernel	`6.15.4-crashnt-4.7_no-built-in-fw` #2, clang 18.1.3, built 2026-03-14	observed
Root	`/dev/loop0` ← `/ps4hdd/home/jupi-linux-2a79fafeb62d61ff.img`, 381,178,347,520 B	observed
Root usage	350 G size, 33 G used, 313 G free (UFS backing: 333 G free)	observed 8:10 AM
Host / user	`playstation` / `pi` (uid 1000, wheel; passwordless sudo)	observed
Locale / TZ	en-GB / Europe/London (from the 2022 psxitarch image)	observed
Current boot	`5c74d073-6f75-42c5-b5d4-566cc6090faa`, up since Sat Nov 7 11:15:16 PM	observed
Network	eth0 /24 (`proto dhcp`), default route via Roach ; no wlan interface up	observed
Unattended cold boot	**Not verified** (`cold_boot_verified: false` in every boot receipt)	observed
Kernel loaded from USB or HDD copy this boot	**UNKNOWN**	—

The three things to remember

1. **Memory is one pool.** The GPU's working buffers (GTT) are carved out of the same 6.7 GiB the apps use. Xorg can hold ~3.4 GiB of it for desktop apps. When that happens the box freezes for new logins, and **only restarting the X session (lightdm) gives the memory back.** See Sections 6–7.
2. **After any X restart the HDMI panel must be re-handshaked** (it reports zero EDID bytes). Five commands, Section 7, step R4.
3. **Keep a pre-opened SSH connection.** During the Nov 10 freeze 37 fresh SSH logins failed while ping still worked. A connection opened *before* the freeze is the one that still works.

2. Origin: the rescue-shell story

JUPI did not start as a plan. It started as a boot that never finished.

2.1 The first boot script, and the typo

Kam's first custom boot hook, `fc.sh`, was meant to decrypt the PS4 drive, mount Sony's UFS2 filesystem at `/ps4hdd`, open the stock 50 GiB `linux.img` read-only and lay a writable overlay from the USB stick over it. Here it is exactly as preserved in rescue checkpoint 02 (sha256 `4c4f902b…2fe6143b`). It is still inside today's initramfs as an inert file:

```
sleep 4;P=/dev/sda27;U=/dev/sdb2;[ -b /dev/sdb27 ]&&P=/dev/sdb27&&U=/dev/sda2
cryptsetup -d <loader key file in initramfs> --cipher aes-xts-plain64 -s 256 --offset 0 --skip 0 create ps4hdd $P
mkdir -p /ps4hdd /ro /rw /newroot && mount -t ufs -o ro,ufstype=ufs2 /dev/mapper/ps4hdd /ps4hdd
losetup -r /dev/loop0 /ps4hss/home/linux.img && mount -t ext4 -o ro,noload /dev/loop0 /ro
                      ^^^^^^^
                      "/ps4hss" -- should be /ps4hdd. The image is never attached.
mount -t ext4 -o rw $U /rw && mkdir -p /rw/u/home /rw/w && mount -t overlay overlay \
      -o lowerdir=/ro,upperdir=/rw/u,workdir=/rw/w /newroot
```

The domino run is mechanical. `losetup` fails on a path that does not exist, so nothing is mounted on `/ro`. The overlay has no lower layer, so `/newroot` stays empty. The initramfs (`better-initramfs`) then cannot mount or `switch_root` into `/newroot`. Its `die` path calls `rescueshell`, and the PS4 sits at a BusyBox prompt. *(Inferred from the init and functions.sh in checkpoint 02. The exact on-screen error is UNKNOWN.)*

> **A second hazard was hiding in the same log.** Checkpoint 01's kernel log shows `/dev/sdb2: Can't lookup blockdev` at **5.45 s**, while Sony's USB-SATA bridge (the internal HDD) did not enumerate until **6.65 s** (after an `error -110` retry). `fc.sh` slept only 4 s, so at the moment it chose `P`/`U` the internal disk did not exist yet. Even with the typo fixed, that boot was racing the hardware. *(Observed timings, inferred consequence.)*

2.2 The rescue-shell records (CP01–CP03)

With no real-time clock inside the initramfs, everything written from the rescue shell is dated **1 Jan 1970**. Kam's notes were saved on the USB root under `/fullcircle/ps4-rescue-inventory/`. Fast copies are on Roach in `~/ps4-linux`.

```
/fullcircle/ps4-rescue-inventory/            (USB sda2, timestamps = 1970, clock not set)
|-- checkpoint-01/   system.txt  boot-hashes.txt  kernel-log.txt  SHA256SUMS   (02:41 "1970")
|-- checkpoint-02/   fc.sh  init  functions.sh                    SHA256SUMS   (02:46-02:48)
`-- checkpoint-03/   initramfs.pre-v3.cpio.gz  initramfs.v3.cpio.gz   SHA256SUMS (04:15-04:31)
/root/ps4boot-work/  initramfs.original / initramfs.fixed / f.gz, unpacked trees t/ tree/ verify/
```

CP01 captured the rescue environment: kernel `6.15.4-crashnt-4.7`, USB `sda2` mounted `ro,norecovery` at `/usb`, `PS4BOOT` at `/ps4boot`, and the filesystem list (which already included `ufs`). The absolute time of CP01–CP03 is **UNKNOWN**. They fall after the USB was prepared (Nov 5, ~6:01–6:50 PM, real-clock files on the stick) and before the first successful boot (Nov 5, 11:45 PM). *(Ordering inferred.)*

2.3 From rescue shell to Arch, step by step

```
Nov 5  ~6-7 PM   USB stick prepared (psxitarch image + PS4BOOT kernel/initramfs)
       (1970)    fc.sh boot -> typo + 4 s race -> RESCUE SHELL; CP01..CP03 written by hand
Nov 5 11:45 PM   Roach sends linux-1024mb.bin (329,088 B) to the PS4 payload port
      11:47 PM   Ethernet returns; OpenSSH 10.6 answers; host key awkaTxgG... first seen
      11:55 PM   Kam confirms: XFCE desktop on the PS4 screen
Nov 6 12:34 AM   Verified root: loop0 <- /ps4hdd/home/linux.img (stock 50 GiB image) on UFS2
      12:41 AM   Autostart files added to the HDD (/data/linux/boot, payload AutoRun)
    by 1:02 PM   Root is the USB stick (sda2) after a reported MANUAL switch_root: the USB
                 selector died on a syntax error -- "[ -x /newroot/sbin/init }"  (missing "]")
Nov 6  1:06 PM   CP05: one-byte fix "}" -> "]", 10/10 selector tests, rollback copies x3
       1:21 PM   Restart test boots USB root automatically (boot b00c61e1)
       8:48 PM   FIRST BOOT ON THE JUPI IMAGE: loop0 <- jupi-linux-....img (40 GiB), boot 54f4c94e
       9:13 PM   Disk-only selector activated on both boot copies
       9:16 PM   Image grown online 40 GiB -> 355 GiB
Nov 7 11:15 PM   Kernel reboot -> boot 5c74d073 (the current boot)
```

2.4 Why the user is `pi` and the clock says London

Root's 2022 shell history on the USB image (`/root/.bash_history`, 429 bytes, dated 26 Jun 2022) shows how the base image was originally set up:

```
useradd pi -m -G wheel
ln -sf /usr/share/zoneinfo/Europe/London /etc/localtime
nano /etc/locale.gen ; locale-gen ; nano /etc/hostname ; nano /etc/sudoers
```

JUPI's image was built from that USB root, so `pi`, `en-GB` and London time came along with it *(inferred: the image still carries the USB root's Nov 6 dotfiles such as `.bash_profile.before-nikolai-20261106T2239Z`)*. The USB image's `/version` reads `2022.06.01`. The machine is a PS4 (`x86_64`), and the Pi-style names are inherited.

3. Boot chain

JUPI's boot runs in two halves. The first half belongs to the PS4: Sony's console OS has to be running before a homebrew payload can hand control to Linux. The second half belongs to Linux: an initramfs root selector decides, with strict checks, whether to boot the JUPI image from the internal HDD or fall back to the old USB root.

3.1 Step diagram

```
[0] POWER ON  ->  Orbis OS (Sony firmware) boots from the internal HDD
      |
[1] Homebrew enabler runs; its AutoRun launches the Linux payload
      |   /data/GoldHEN/payloads.ini AutoRun -> /user/data/payloads/linux-1024mb.bin (329,088 B, e11da9d2...)
      |   "1024mb" = the loader variant that reserves 1 GiB of the shared pool as GPU VRAM
      |   Whether this step needs a human at the console each boot: UNKNOWN
      v
[2] Payload loads bzImage + initramfs and starts Linux
      |   copies: USB PS4BOOT (sda1)  or  HDD /ps4hdd/data/linux/boot/   -> which one: UNKNOWN
      |   both copies: kernel f0008d46...  initramfs cdeaefdb...  (same bytes)
      |   the loader hands the HDD unlock key to Linux as a 32-byte file in the initramfs
      v
[3] Kernel 6.15.4-crashnt-4.7 starts
      |   cmdline: panic=0 clocksource=tsc consoleblank=0 net.ifnames=0 radeon.dpm=0
      |            amdgpu.dpm=0 drm.debug=0 console=ttyS0,115200n8 console=tty0
      |            video=HDMI-A-1:1920x1080@60
      v
[4] better-initramfs /init  ->  ". /usbroot.sh"   (line 97; it replaced ". /fc.sh")
      |
      |   +------------------------ ROOT SELECTOR  usbroot.sh  (sha256 f65941df...) ----------+
      |   | sleep 5                                                                          |
      |   | any UFS / mapper / /newroot already present? ---------------- yes -> HOLD (rc 2)   |
      |   | key file present and exactly 32 bytes? ----------------------- no -> USB  (rc 1)   |
      |   | exactly one /dev/sdX whose GPT header + partition-27 GUID match? 0 -> USB (rc 1)   |
      |   |                                                    more than 1 -> HOLD (rc 2)    |
      |   | cryptsetup create jupi-boot-internal (plain, aes-xts-plain64, 256, offset 0)     |
      |   | mount UFS2 READ-ONLY at /ps4hdd (nosuid,nodev,noexec)                            |
      |   | image file present, size = 40 GiB or 355 GiB exactly?                            |
      |   | losetup -r ; blkid: UUID 70dbf870... and TYPE ext4?                              |
      |   | mount ext4 READ-ONLY,noload ; /usr/local/bin/jupi sha256 == pinned 3f5d4eee... ;   |
      |   |   /sbin/init resolves inside /newroot and is executable                          |
      |   | unmount, detach  ->  remount UFS2 READ-WRITE  ->  losetup RW  ->  recheck UUID     |
      |   | mount ext4 RW at /newroot ; recheck content ; mount --move /ps4hdd /newroot/ps4hdd |
      |   | print  JUPI_INTERNAL_ROOT_SELECTED_BOOT_UNPROVEN                    -> rc 0        |
      |   | any failure: JUPI_INTERNAL_UNAVAILABLE_<STEP>, clean teardown -> rc 1 (USB)        |
      |   |              teardown itself fails -> JUPI_HOLD_CLEANUP          -> rc 2 (HOLD)    |
      |   +----------------------------------------------------------------------------------+
      |
      |   rc 0 -> continue with JUPI image            rc 2 -> "JUPI_HOLD_INTERNAL_STATE", /bin/sh
      |   rc 1 -> legacy USB selector: mount the "psxitarch" ext4 on sda2/sdb2 RW
      |           HOLD_NO_USB / HOLD_MOUNT / HOLD_INIT -> /bin/sh
      v
[5] Key-retention hook: if absent, copy the loader's key to a root-only file on the new root
      |   (<sealed root-only copy on the image>, mode 600, cmp-verified)
      |   prints JUPI_CONSOLE_KEY_RETAINED / _ALREADY_RETAINED / _DIFFERENT_EXISTING_FILE_PRESERVED
      v
[6] switch_root /newroot  ->  systemd  ->  multi-user.target + jupi-platform.target
      |   ps4hdd backing stays mounted at /ps4hdd (UFS2 rw, onerror=lock)
      v
[7] lightdm  ->  autologin pi  ->  XFCE  ->  ghostie-display-keeper keeps DPMS off
```

3.2 Selector exit paths at a glance

Selector result	Console message(s)	What boots	Operator meaning
rc 0	`JUPI_INTERNAL_ROOT_SELECTED_BOOT_UNPROVEN`	JUPI image (HDD)	Normal
rc 1	`JUPI_INTERNAL_UNAVAILABLE_<STEP>` or none (no key / no disk)	**Old USB root** `sda2` *psxitarch*	Silent fallback. Looks like JUPI is up, but it is the 2022-based USB system
rc 1, no USB	`HOLD_NO_USB`, `HOLD_MOUNT`, `HOLD_INIT`	BusyBox shell	Rescue shell (Section 2 déjà vu)
rc 2	`JUPI_HOLD_EXISTING_STATE`, `_AMBIGUOUS_PARTITION`, `_CLEANUP`, `_LOOP_STATE`, `_BACKING_LOCATION` … then `JUPI_HOLD_INTERNAL_STATE`	BusyBox shell	Selector refused to guess. Read the message, do not force

`<STEP>` is one of `UFS_RO, IMAGE, FREE_LOOP, LOOP_NAME, LOOP_RO, IDENTITY, ROOT_RO, ROOT_CONTENT, ROOT_RO_RELEASE, LOOP_RO_RELEASE, UFS_RW, LOOP_RW, IDENTITY_RW, ROOT_RW, ROOT_CONTENT_RW, BACKING_MOVE`.

> **Watch the silent fallback.** If the HDD bridge enumerates late, the GPT check finds no disk and the selector takes rc 1. JUPI then boots the *USB* root without complaint. In the rescue era the bridge appeared at 6.65 s, and the selector waits only 5 s. Whether 5 s is always enough on this hardware is **UNKNOWN**. After every reboot, confirm `findmnt -no SOURCE /` prints `/dev/loop0` (Runbook R6).

3.3 Design notes worth stealing

- **Two-phase mount.** Every check runs on a read-only mount first (`ro,noload`, no journal replay). Only then is the UFS remounted read-write and the image reattached read-write. A wrong image is never opened writable.
- **Identity pinning.** The selector accepts the image only if three things match: an exact byte size, the ext4 UUID, and the sha256 of a specific binary inside it (`/usr/local/bin/jupi`). That binary is why the README says it must stay *byte-for-byte* unchanged.
- **Refuse rather than guess.** Pre-existing state, two candidate disks, or a failed teardown all stop at a shell (rc 2) instead of picking one.
- **Symlink paranoia.** Every path test also rejects symlinks (`jupi_block`, `jupi_file`, `jupi_dir`).

3.4 What is proven and what is not

Claim	Status	Evidence
Internal root selected and booted	Proven on 2 kernel boots (54f4c94e, 5c74d073)	boot receipts
Same kernel/initramfs bytes on USB and HDD copies	Proven on disk	`disk-boot-activation.json`, hashes
Which copy the loader read this boot	**UNKNOWN**	receipt: "loaded_boot_file_bytes UNKNOWN"
USB stick physically absent at boot	**Untested**	`usb_absent_boot_test: pending`
Unattended return after total power loss	**Not verified**	`cold_boot_verified: false` ×3
Userspace soft-reboot	Proven once, Nov 7 (SoftRebootsCount 0→1)	`/etc/jupi/AGENTS.md`

Claim	Status	Evidence
kexec warm reboot	**Held**: early GPU firmware set incomplete	`native-warm-qualification.json`
Serial console ttyS0 as recovery path	**Not usable**: UART init disabled in source	`/etc/jupi/AGENTS.md`

4. Storage layout

```
sda  SanDisk USB, 114.6 GiB, DOS label 0xeac72143            (boot medium + fallback root)
|-- sda1   512 MiB  vfat  PS4BOOT    891E-578E    kernel + initramfs (USB boot copy)
|-- sda2 106.1 GiB  ext4  psxitarch  2ec1035f...  OLD root (fallback), not mounted
`-- sda3     8 GiB  swap  ps4swap    7762469a...  retired Nov 6 9:17 PM (commented in fstab)

sdb  Internal HDD via Sony USB-SATA bridge, 465.8 GiB, GPT 66FB93BD-..., 15 partitions
|-- sdb27 420.1 GiB  --dm-crypt plain aes-xts-plain64-->  /dev/mapper/jupi-boot-internal
|                                                          `-- UFS2 (Sony)  mounted at /ps4hdd
|                                                              414 G size, 48 G used, 333 G free
|     /ps4hdd/home/jupi-linux-2a79fafeb62d61ff.img   381,178,347,520 B (355 GiB) -> /dev/loop0 = /
|     /ps4hdd/home/linux.img                          53,687,091,200 B (50 GiB, stock image, idle)
|     /ps4hdd/data/linux/boot/{bzImage, initramfs.cpio.gz, initramfs.before-disk-only-...}
|     /ps4hdd/data/{GoldHEN, payloads, sce_logs, ...}  + Sony system dirs (appmeta, system, ...)
`-- sdb1..sdb31 (14 other Sony partitions, unknown types) -- never touched

zram0  2 GiB  lz4   swap  JUPI_ZRAM   prio 100   (jupi-zram.service)
zram1  2 GiB  zstd  swap  groku-zram  prio 200   (groku-zram.service, used first)
```

Mount	Source	Type / options	Size / used / free
/	/dev/loop0	ext4 rw,relatime	350 G / 33 G / 313 G
/ps4hdd	/dev/mapper/jupi-boot-internal	ufs rw,nosuid,nodev,noexec,ufstype=ufs2,onerror=lock	414 G / 48 G / 333 G
/tmp	tmpfs	3.4 G	113 M used
/run/user/1000	tmpfs	691 M	—

The image is sparse. Its free space is borrowed. The root reports 313 GiB free. Those blocks do not exist until written, and they come out of the UFS's 333 GiB. `/etc/jupi/boot.json` keeps a **16 GiB backing headroom** (`backing_headroom_bytes: 17179869184`). Do not grow another image into that reserve. If the UFS fills, the root filesystem fails underneath a running system.

Do not touch the 14 other Sony partitions, `/ps4hdd/system`, `/ps4hdd/home/<ps4-user-id>` (the console user profile) or the Orbis-side files. They are what make step [0]–[1] of the boot work.

5. As-built OS configuration

All values were read from the 8:10 AM scan unless stated otherwise. Arch Linux rolling, 844 packages installed (`pacman` count line also shows 150, most likely the explicitly-installed subset).

5.1 Kernel and sysctl

Key	Live value	Set by
`vm.swappiness`	**100**	`/etc/sysctl.d/90-groku.conf` (overrides `80-jupi-memory.conf` = 10)
`vm.page-cluster`	**0**	`90-groku.conf` (single-page swap-in; right for zram)
`vm.min_free_kbytes`	67584	kernel default for this RAM size
`vm.watermark_scale_factor`	10	default
`vm.vfs_cache_pressure`	100	default
`vm.overcommit_memory`	0 (heuristic)	default; Committed_AS 11.2 GB vs CommitLimit 7.7 GB
SysRq	**absent** (`/proc/sys/kernel/sysrq` does not exist)	kernel build
`CONFIG_CFS_BANDWIDTH`	**absent**, so `CPUQuota=` is not enforced; CPU affinity is used instead	kernel build

Why swappiness 100? All swap is compressed RAM (zram). Swapping an idle anonymous page costs a fast compression. Dropping a file page means re-reading it later from a loop image on an encrypted HDD behind a USB bridge. *(The `90-groku.conf` comment still says "USB loop root". That wording is stale. The idea still holds for the HDD loop root.)*

5.2 Swap: two zram devices

Device	Algorithm	Size	Priority	Owner unit	Script sha256
`/dev/zram1` `groku-zram`	zstd	2 GiB	**200** (used first)	`groku-zram.service` (After=jupi-zram)	`e6d2ca3f…6c52bd`
`/dev/zram0` `JUPI_ZRAM`	lz4	2 GiB	100	`jupi-zram.service` (Before=lightdm)	`8d9d1f4f…a440f2`

At scan: zram0 held 164.9 MiB of data in 73.8 MiB of RAM; zram1 held 15.2 MiB in 3 MiB. The old 8 GiB USB swap partition is commented out of `/etc/fstab`. There is no disk swap.

5.3 earlyoom (installed Nov 10, armed 6:25 AM)

```
/usr/bin/earlyoom -m 4,2 -s 10,5 -r 300
   --prefer '^(llama-bench|llama-cli|llama-server|llama-simple|test-backend-op|grok-bot|chatgpt|ChatGPT|codex)$'
   --avoid  '^(sshd|sshd-session|musegadget|runtime\.py|Xorg|systemd|systemd-journal|systemd-logind|
             xfwm4|xfce4-session|earlyoom)$'
   (drop-in /etc/systemd/system/earlyoom.service.d/90-groku.conf, OOMScoreAdjust=-1000)
```

Trigger	Condition (BOTH must hold)	In absolute terms on JUPI
SIGTERM	MemAvailable ≤ 4 % **and** swap free ≤ 10 %	≤ ~276 MiB available and ≤ ~410 MiB swap free
SIGKILL	MemAvailable ≤ 2 % **and** swap free ≤ 5 %	≤ ~138 MiB available and ≤ ~205 MiB swap free

Package `earlyoom 1.9.0-1`, installed with `pacman -U` (sha256 `f54e18d5…3bba59`). It moves the `jupi-profile` baseline off its pinned state. No kill events were logged up to the scan.

OOM-score protections (lower = kept alive longer):

Process	oom_score_adj	Source
sshd	−1000	pre-existing
earlyoom	−1000	drop-in
jupi-agent (`runtime.py`)	−900	`jupi-agent.service.d/90-groku-oom.conf`
musegadget	−900	`musegadget.service.d/90-groku-oom.conf`

5.4 systemd: what runs

`jupi-platform.target` (WantedBy multi-user) pulls in: `jupi-agent`, `jupi-health.timer`, `jupi-zram`, `jupi-link@roach`, `jupi-firewall`, `sshd`, `musegadget`, `jupi-nerves.timer`, `fclink`, `jupi-link@fclink`, `jupi-observe.timer`, `ollama`.

Service	Runs as	Limits / notes
`jupi-agent.service`	pi, nice 10	MemoryHigh 768M, MemoryMax 1536M, CPUAffinity **0 2**, TasksMax 96; durable SQLite job queue
`ollama.service`	ollama, nice 10	127.0.0.1:11434, 1 model, 1 parallel, ctx 2048, keep-alive 60 s, `OLLAMA_NO_CLOUD=1`, MemoryHigh 1600M / Max 2300M, all 8 CPUs
`musegadget.service`	**root**, commands as pi	Muse Home Link; RuntimeDirectory `/run/musegadget`; Restart on-failure
`jupi-link@roach` / `@fclink`	pi	persistent outbound SSH to Roach (Section 8), Restart always 15 s
`fclink.service`	(unit is root-only, not readable)	FC-Link v1, HMAC, binds + 127.0.0.1 :8765
`jupi-firewall.service`	root, oneshot	loads `/etc/jupi/firewall.nft` (table `inet jupi`)
`jupi-zram` / `groku-zram`	root, oneshot	the two swap devices
`jupi-boot-record.service`	root, oneshot	startup receipt; requires agent + ollama + zram
`earlyoom.service`	dynamic user	see 5.3
`lightdm.service`	root	autologin `pi`, session `xfce`

Timer → service	Cadence (from timer listing)	What it does
`ghostie-usb-guardian`	~1 min	revives a dead xHCI controller, pins hub power, logs churn
`jupi-nerves`	~1 min	local "nervous-system" pulse (timeout 20 s, MemoryMax 128M)
`jupi-host-watch`	~1 min	bounded host resource/service sample (timeout 15 s)
`ghostie-display-keeper`	~2 min	`xset dpms 0 0 0; s off; s 0 0; s noblank; -dpms`
`jupi-observe`	~5 min	hardware + cluster observation (CPUQuota 50 %, idle I/O)
`jupi-knowledge`	~15 min	private archive compile + evidence probes (timeout 300 s)
`jupi-health`	~15 min	inventory + health receipt → `/var/lib/jupi/health.json`

At scan time `jupi-health.service` was **failed**, along with one `systemd-coredump@` unit. Since the journal's oldest entry (Nov 9, 7:54 AM) the observers have logged many timeouts: jupi-nerves 201, jupi-host-watch 196, jupi-observe 45, jupi-knowledge 16, jupi-health 16. When those happened is not broken down. *(Inferred: most likely during memory pressure.)*

Sleep, suspend, hibernate, hybrid-sleep and suspend-then-hibernate targets are masked (symlinked to `/dev/null`). No cron is installed. Systemd timers are the only scheduler.

5.5 Display stack

Piece	Setting
Kernel connector	`card0-HDMI-A-1`, status connected, **EDID 0 bytes**, cmdline forces `1920x1080@60`
X output name	**`HDMI-A-0`** (xf86-video-amdgpu numbers from 0; the kernel numbers from 1)
Xorg	`/usr/lib/Xorg :0 -seat seat0 -auth /run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch`
Driver stack	`xf86-video-amdgpu-ps4 21.0.0-2`, `mesa-ps4 21.3.2-1` (+lib32), LLVM 13 compat libs in `/usr/local/lib/jupi-gpu-compat`
DPMS / blanking	Off. `/etc/X11/xorg.conf.d/10-ghostie-nodpms.conf` (root-only) + the display-keeper timer
Session	lightdm → autologin `pi` → XFCE; `~/.xsession-errors` collects all X client stderr

> `-novtswitch` means **Ctrl+Alt+F*n* will not take you to a text console.** Together with the missing SysRq, a frozen desktop cannot be escaped from the local keyboard.

5.6 Journal, firewall, packages

journald: persistent, `SystemMaxUse=128M`, `SystemKeepFree=1G`, 14-day retention, sync every 5 min. It was at 126.3 MiB at scan, so only **Nov 9 7:54 AM onward** is still in the journal. Older history comes from receipts.

nftables (`table inet jupi`, input policy **drop**):

```
iifname "lo" accept
ct state established,related accept ; ct state invalid drop
meta l4proto { icmp, ipv6-icmp } accept
udp 67->68, udp 547->546 accept          # DHCP renewal
tcp dport 22 accept                      # SSH, any source
iifname "eth0" ip saddr { <lab-gateway-ip>, <roach-lan-ip> } tcp dport 8765 accept   # signed FC-Link
counter drop
```

Foreign (non-repo) packages: `mesa-ps4 21.3.2-1`, `lib32-mesa-ps4 21.3.2-1`, `xf86-video-amdgpu-ps4 21.0.0-2`, `lib32-libvdpau 1.5-1`, `neofetch`, `python-nspektr 0.4.0-1`, `yay-bin 11.2.0-1`. `pacman.conf` has **no `IgnorePkg`** line. A blanket `pacman -Syu` is not fenced off from the PS4 graphics stack. The kernel is not a pacman package, so pacman cannot replace it.

sshd (drop-in): `PermitRootLogin no`, `PasswordAuthentication no`, `KbdInteractiveAuthentication no`, `PubkeyAuthentication yes`, `MaxAuthTries 3`, `LoginGraceTime 30`, `ClientAliveInterval 60` ×3, `X11Forwarding no`, `GatewayPorts no`, `LogLevel VERBOSE`.

6. Unified memory model

A PC has RAM for the CPU and separate VRAM on the graphics card. A PS4 does not. CPU and GPU share one pool of GDDR5. On JUPI that means **a desktop app's graphics buffers come out of the same memory the kernel, the agent and SSH need to survive.**

6.1 Where the 8 GiB goes

```
PHYSICAL GDDR5 (8 GiB, PS4 spec) -- one pool
+--------------------+-----------------+-----------------------------------------------------+
| firmware / Sony    | GPU "VRAM"      | LINUX RAM  (MemTotal 7,070,228 kB = 6.74 GiB)       |
| reserved: UNKNOWN  | carve-out 1 GiB |                                                     |
| exact split        | (1024mb loader) |                                                     |
+--------------------+-----------------+-----------------------------------------------------+
                                        |
                   inside Linux RAM     v
+---------------+-------------+--------------+-------------------------------------------+
| app memory    | page cache  | kernel slab  | GTT: system RAM the GPU maps              |
| (anon, swap-  | (reclaimable| ~1.3 GiB     | cap 3,618,242,560 B = 3.37 GiB            |
|  pable to     |  instantly) | (1.21 GiB    | = half of MemTotal (amdgpu default sizing) |
|  zram)        |             |  unreclaim.) | buffers held by Xorg for X clients do NOT  |
|               |             |              | come back under pressure                  |
+---------------+-------------+--------------+-------------------------------------------+
swap: 4 GiB of zram -- it also lives in this same RAM (compressed)
```

Every layer competes for the same 6.74 GiB. Swap brings no new memory. zram only compresses what is already in RAM. So once the GTT holds 3.4 GiB, everything else (the kernel, sshd, the agent, the page cache that holds your shell binaries) fights over about 3.3 GiB, and zram fills too.

6.2 Anatomy of the Nov 10 freeze

```
TIME (PST)  AVAILABLE   SWAP USED      GTT USED / CAP          WHAT HAPPENED
----------  ---------   ------------   ---------------------   ---------------------------------
~6:2x AM     ~190 MB    4024/4095 MB   3,615,711,232 / 3.62 G  GTT at 99.93 % of cap (Xorg)
7:02-7:17         --              --   (unreachable)           37 fresh SSH logins fail; ping OK
7:31-7:36    107 MB     4095/4095 MB   --                      Grok Bot app killed: NO relief
7:43:33      100 MB     4095/4095 MB   --                      still pinned
7:47:03    -------- sudo systemctl restart lightdm (pi, from a session that got in) --------
7:50:01     4343 MB      183/4095 MB   --                      RAM back
7:50:18     4076 MB      183/4095 MB   32,657,408 (31 MiB)     GTT released
8:10        3135 MB      183/4095 MB   31,875,072 (30 MiB)     as-built scan
8:24        3111 MB      199/4095 MB   31,875,072 (30 MiB)     this manual's spot check
```

Three lessons, all from the record above:

1. **Killing the app is not enough.** The 3.4 GiB was attributed to **Xorg**, which holds the buffers on behalf of its clients. Killing the Grok Bot desktop app at 7:31–7:36 left available memory at 107 MB.
2. **Restarting the X session is the release valve.** When lightdm stopped (7:47:28), Xorg exited and the kernel freed its GTT. lightdm's own unit summary reads *"614.9M memory peak, **3.4G memory swap peak**"*.
3. **earlyoom cannot fix this on its own.** It avoids `Xorg` by design and kills only apps (ChatGPT, grok-bot, codex, llama-*). That buys time but cannot release Xorg's hold. Use it as the seatbelt and the lightdm restart as the brake.

6.3 The health gauge: always read two numbers

`free -m` alone lies by omission, because GTT shows up as "used" with no owner. Read **MemAvailable and GTT together**:

```
free -m
g=/sys/class/drm/card0/device
echo "GTT $(( $(cat $g/mem_info_gtt_used)/1048576 )) / $(( $(cat $g/mem_info_gtt_total)/1048576 )) MiB"

# who holds it (root; per-process sum of GTT buffer objects from the amdgpu debug node)
sudo awk '/^pid/{p=$2" "$4} /byte GTT/{s[p]+=$2}
  END{for(k in s) printf "%8.1f MiB  %s\n", s[k]/1048576, k}' \
  /sys/kernel/debug/dri/0/amdgpu_gem_info | sort -rn | head
```

Healthy output at 8:24 AM: `GTT 30 / 3450 MiB`; holders `Xorg 11.2`, `xfwm4 10.0`, `ChatGPT 2.0`, `xfce4-session 2.0` MiB.

Zone	MemAvailable	GTT used	Swap used	Action
GREEN	> 1500 MiB	< 1024 MiB	< 50 %	Carry on
AMBER	500–1500 MiB	1024–2500 MiB	50–90 %	R2: shed desktop apps, plan an X restart
RED	< 500 MiB	> 2500 MiB	> 90 %	R3 now, while you still have a session

Zone thresholds are operator guidance proposed in this manual, not measured limits. The measured failure point was ~100–190 MB available, 4095/4095 swap, GTT 99.9 % of cap.

6.4 Two operating profiles

These are **operator conventions**. No JUPI config file encodes them (checked at scan).

DESKTOP (interactive / light)	COMPUTE (heavy work)	
Desktop apps	XFCE + at most one heavy Electron app (ChatGPT *or* Grok Bot)	Close ChatGPT, Grok Bot, browsers. Or restart X and launch nothing
Allowed jobs	editing, light agent jobs, qwen3:0.6b prompts	ollama bigger models, `jupi-pipeline-bench`, GPU/UMA benches, builds
Budget ceilings already enforced	jupi-agent ≤ 1536 MiB, ollama ≤ 2300 MiB (systemd MemoryMax)	same; GTT then free for compute
Gauge cadence	every few hours, and before starting anything big	before and after each heavy job
Exit	X restart (R3 + R4) if GTT trends up for hours	relaunch desktop apps afterwards

Rule of thumb (inferred from the incident): GTT climbs slowly while Electron apps sit in the desktop. Watch the trend. A single reading will not show it.

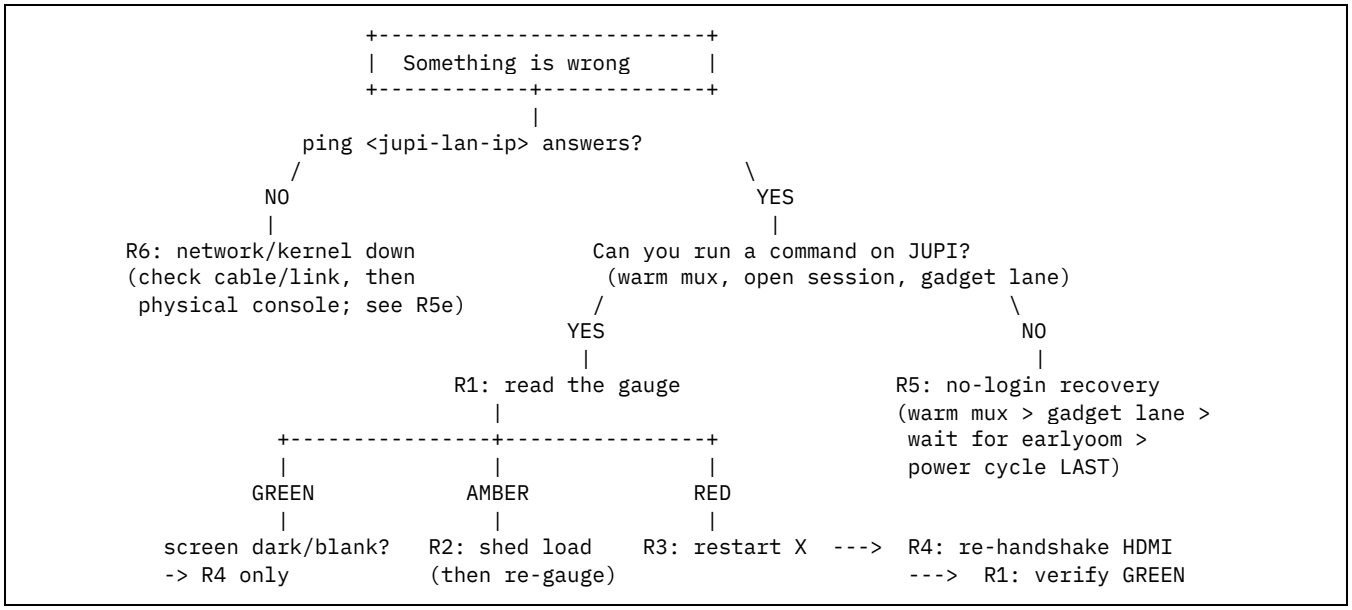
6.5 Other memory observations at scan

- **Unreclaimable slab: 1,210,648 kB (≈1.15 GiB).** That is 17 % of MemTotal, and the cause is **UNKNOWN** (candidates: zram/TTM bookkeeping, driver allocations). Watch whether it grows.
- `Committed_AS` 11.2 GB on a 7.7 GB CommitLimit. Fine under heuristic overcommit, but allocations are promised beyond what physically exists.
- VRAM carve-out: 1,073,741,824 B total, 109 MB used. The GPU uses GTT for the rest.

7. Freeze & recovery runbook

Work top to bottom. Every command was used on JUPI on Nov 10 or read from its live configuration. Run commands as `pi` unless shown with `sudo` (pi has passwordless sudo).

7.0 Triage: pick your entry point



R0 · Pre-arm (do this while healthy, on Roach)

The one thing that worked in a freeze was a session that **already existed**. A fresh SSH login needs sshd to fork, allocate and complete a key exchange, which is what failed 37 times between 7:02 and 7:17 AM. A multiplexed master connection skips all of that for every later command.

`~/.ssh/config` on Roach. *Not present today: Roach has no ControlMaster entry for JUPI (checked at 8:24 AM).*

```
Host jupi
    HostName <jupi-lan-ip>
    User pi
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
    ControlMaster auto
    ControlPath ~/.ssh/cm-%r@%h:%p
    ControlPersist yes
    ServerAliveInterval 30
    ServerAliveCountMax 6
```

```
ssh -fN jupi                # open the master once, in the background
ssh -O check jupi           # -> "Master running (pid=...)"
ssh jupi 'uptime'           # rides the master: no new login on JUPI
```

The reverse tunnel is a second way in that does not depend on JUPI's LAN address: `ssh -p 22052 pi@127.0.0.1` on Roach (JUPI's `jupi-link@roach` forwards Roach's loopback 22052 to JUPI's sshd). It still needs a fresh key exchange on JUPI, so it does **not** survive a freeze by itself. Put a ControlMaster on it too if you use it.

R1 · Read the gauge (30 seconds)

```
ssh jupi 'free -m; swapon --show
  g=/sys/class/drm/card0/device
  echo "GTT $(( $(cat $g/mem_info_gtt_used)/1048576 )) / $(( $(cat $g/mem_info_gtt_total)/1048576 )) MiB"
  ps -eo pid,rss,comm --sort=-rss | head -8'

ssh jupi "sudo awk '/^pid/{p=\$2\" \"\$4} /byte GTT/{s[p]+=\$2}
  END{for(k in s) printf \"%8.1f MiB  %s\n\", s[k]/1048576, k}' \
  /sys/kernel/debug/dri/0/amdgpu_gem_info | sort -rn | head -5"
```

Classify with the zone table (Section 6.3). **RED = MemAvailable < 500 MiB or GTT > 2500 MiB or swap

> 90 %.** If Xorg is the top GTT holder in RED, go straight to **R3**.

R2 · Shed load (AMBER)

Close heavy desktop apps gracefully first. Use the app's own quit if someone is at the screen, or:

```
ssh jupi 'pkill -TERM -x ChatGPT; sleep 10; pgrep -a -x ChatGPT || echo "ChatGPT closed"'
```

> **Caution. `grok-bot` is also a lane.** The Grok Bot desktop app on JUPI hosts a local-exec daemon (`~/.grokbot/local-exec-daemon*.json`). Closing it may cut that agent's access. Close it only knowingly.

Expect: app RSS comes back and swap drains a little. **The GTT does not.** If GTT stays above ~2500 MiB after the apps exit, the hold is in Xorg. Go to R3.

R3 · Release the pin: restart the X session

> **WARNING — what this kills.** Everything in the graphical session: ChatGPT/Codex desktop, Grok Bot app, browser windows, XFCE terminals, unsaved work. Services (`jupi-agent`, `ollama`, `musegadget`, `sshd`, the SSH links) are **not** affected. Never run this as a reflex from an automated loop, and check nobody else already issued it. On Nov 10 two operators raced.

```
ssh jupi 'sudo systemctl restart lightdm'
```

Observed timing on Nov 10: issued 7:47:03 → stopped 7:47:28 → started 7:47:46 → pi autologin 7:49:27. **Allow about 2.5 minutes** before judging. Then:

```
ssh jupi 'systemctl is-active lightdm; pgrep -a Xorg; loginctl list-sessions --no-legend | grep -c .'
```

Then **always** run R4, because the panel will not come back by itself.

R4 · Re-handshake the HDMI panel (after every X restart, or a black screen)

The panel reports **zero EDID bytes**, so after Xorg restarts it may not light up or may pick a wrong mode. Force the mode explicitly. The X output is **`HDMI-A-0`**. (The kernel calls the same connector `card0-HDMI-A-1`. Do not mix them up.)

```
ssh jupi 'export DISPLAY=:0 XAUTHORITY=/home/pi/.Xauthority
  xset dpms force on
  xset -dpms
  xset s off
  xrandr --output HDMI-A-0 --off
  xrandr --output HDMI-A-0 --mode 1920x1080 --rate 60 --primary
  xrandr --query | grep HDMI-A-0
  xset q | grep -E "DPMS is|timeout"'
```

Expect:

```
HDMI-A-0 connected primary 1920x1080+0+0 (normal left inverted right x axis y axis) 0mm x 0mm
  timeout:  0    cycle:  0
  DPMS is Disabled
```

`0mm x 0mm` is normal here (no EDID). If you get `Can't open display`, autologin has not finished yet. Wait 60 s. Or use the server's own auth file as root: `sudo env DISPLAY=:0 XAUTHORITY=/run/lightdm/root/:0 xrandr --output HDMI-A-0 --mode 1920x1080 --rate 60 --primary`.

Then re-run **R1**. **Expect:** GTT ≈ 30–40 MiB, MemAvailable > 3000 MiB, swap draining (Nov 10: 4095 → 183 MiB in under 3 minutes).

R5 · No login possible, ping still answers

Try in this order:

Path	Notes	
a	Warm mux from R0	`ssh -O check jupi` then `ssh jupi 'sudo systemctl restart lightdm'`, then R4
b	Any session already open	e.g. a long-running `ssh` or tmux someone left attached. Use it for R3
c	Gadget lane (Muse Home Link, node `homelink-3364d7`)	`musegadget` is an already-running root service that runs commands as `pi` (`system.run`, socket `/run/musegadget/musegadget.sock`). It needs no new SSH login. Whether it stays responsive in a deep freeze is **UNKNOWN** (not exercised on Nov 10). The exact client invocation is outside this manual
d	Keep retrying SSH at a slow cadence	On Nov 10 a login got through at ~7:46 AM after 45 min of pressure. Use `ConnectTimeout=25`. Do not run parallel retry loops from several machines, because they compete for the same sshd
e	Wait for earlyoom	Fires only when available ≤ ~276 MiB **and** swap free ≤ ~410 MiB. Kills ChatGPT / grok-bot / codex / llama-* first. **Cannot release Xorg's GTT**, so it buys minutes, not a fix
f	Local keyboard on JUPI	Only if the XFCE desktop still responds: open a terminal and run R3. **Ctrl+Alt+F*n* does nothing** (`-novtswitch`). **No SysRq** in this kernel
g	Power cycle: **LAST RESORT**	See the warning below

> **WARNING — power cycling JUPI is not a reboot button.** Returning to Linux needs the console OS and the homebrew loader to run first (Section 3, steps 0–2). Unattended return after power loss is **not verified**. Someone must be physically at the PS4 and ready to launch the payload. A hard power cut also lands on a running ext4 image inside a UFS2 filesystem. Afterwards, run R6 before trusting the box.

R6 · After any reboot: prove you are on the right root

```
findmnt -no SOURCE,FSTYPE /          # expect: /dev/loop0 ext4
findmnt -no SOURCE,FSTYPE /ps4hdd    # expect: /dev/mapper/jupi-boot-internal ufs
losetup -l | grep jupi-linux         # expect: /ps4hdd/home/jupi-linux-2a79fafeb62d61ff.img
uname -r                             # expect: 6.15.4-crashnt-4.7_no-built-in-fw
cat /proc/sys/kernel/random/boot_id  # new boot -> new id; record it
swapon --show                        # expect: zram1 zstd prio 200 + zram0 lz4 prio 100
sysctl vm.swappiness vm.page-cluster # expect: 100 / 0
systemctl is-active earlyoom jupi-agent musegadget sshd lightdm jupi-link@roach fclink
systemctl --failed --no-legend
```

> **If `/` is `/dev/sda2` (label *psxitarch*)** the selector fell back (rc 1). You are on the old USB-era system, not JUPI. Do not run JUPI installers or restore jobs against it. Note any `JUPI_INTERNAL_UNAVAILABLE_*` line from the boot console and stop for review.

R7 · Undo the Nov 10 memory changes (only if instructed)

Every Nov 10 change was additive and has a rollback:

```
# review first
sed -n '1,80p' /home/pi/groku-wire/rollback-step1.sh
sha256sum /home/pi/groku-wire/rollback-step1.sh
#   expect 372417fcd4bdc09fbfb76d9b98c577ebc8add9348665cded87479c36ffef2303
bash /home/pi/groku-wire/rollback-step1.sh    # zram1, 90-groku sysctl, OOM drop-ins, earlyoom drop-in
sudo pacman -R earlyoom                         # removes the package itself
```

Pre-change copies: `/home/pi/groku-wire/backup-20261110T135318Z/`. Change log (hash-chained): `/home/pi/groku-wire/CHANGES.md`.

R8 · Housekeeping: the runaway log (current open issue)

`~/.xsession-errors` is growing ~175–183 KB/s (≈15 GB/day). The cause is ChatGPT's GPU process looping on `SharedContextState::InitializeGL failure` (Section 10.1). When an operator approves:

```
ssh jupi 'pkill -TERM -x ChatGPT'                # stop the writer (see R2 caution)
ssh jupi 'truncate -s 0 ~/.xsession-errors; ls -l ~/.xsession-errors'
```

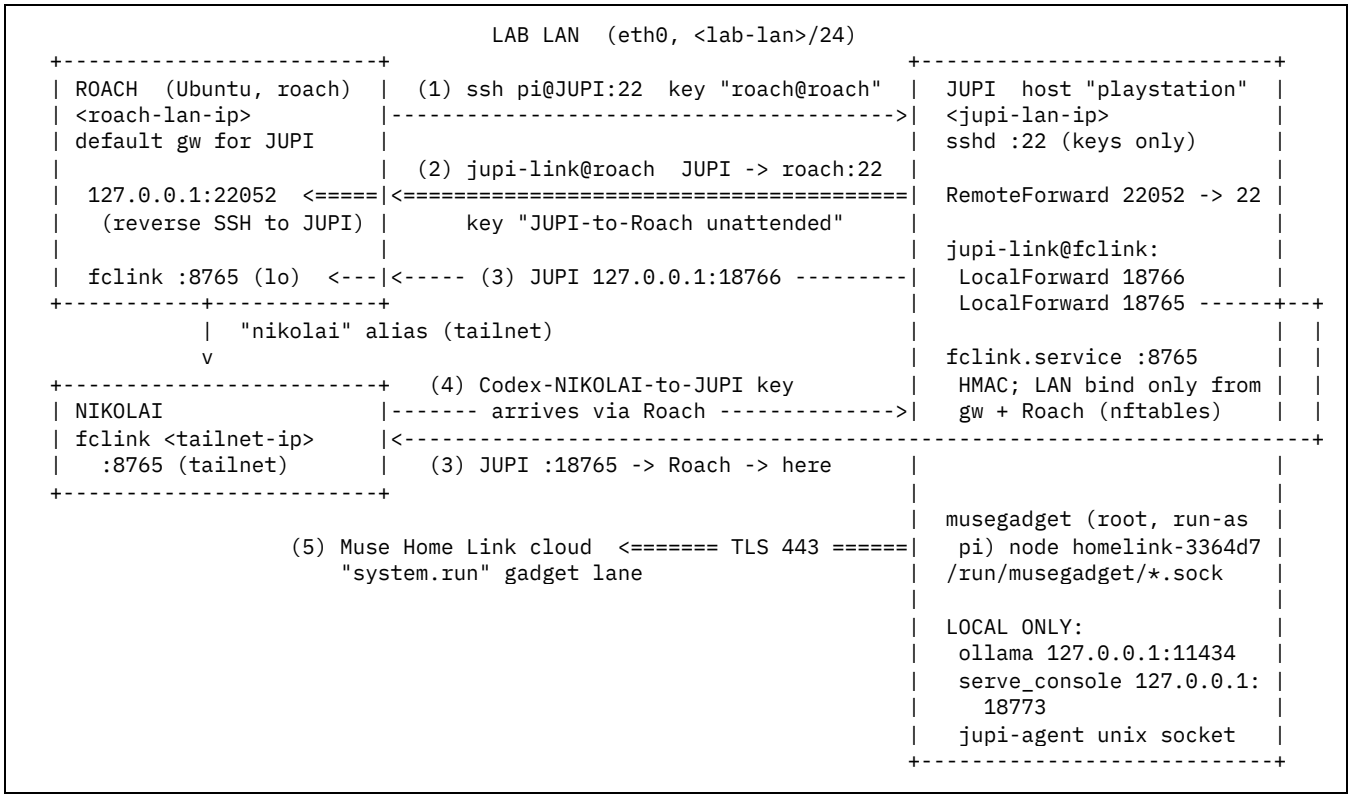
Not executed while this manual was written. JUPI was kept read-only.

Quick card (cut out and keep)

```
GAUGE   free -m ; GTT = /sys/class/drm/card0/device/mem_info_gtt_{used,total}
RED     avail < 500 MiB | GTT > 2500 MiB | swap > 90 %
FIX     sudo systemctl restart lightdm          (kills desktop apps, not services)
THEN    export DISPLAY=:0 XAUTHORITY=/home/pi/.Xauthority
        xset dpms force on; xset -dpms; xset s off
        xrandr --output HDMI-A-0 --off
        xrandr --output HDMI-A-0 --mode 1920x1080 --rate 60 --primary
VERIFY  GTT ~30-40 MiB, avail > 3000 MiB, "DPMS is Disabled", findmnt / = /dev/loop0
NEVER   power-cycle as a first move; it needs a human at the console to come back
```

8. Lanes & access map

8.1 Map



8.2 Lane table

#	Lane	Direction / transport	Auth	Notes
1	**Direct SSH**	Roach → JUPI tcp/22	`pi` authorized_keys line 1: `roach@roach`	The lane used for the scan and this manual. Fresh logins failed in the freeze (R0)
2	**Reverse tunnel**	JUPI → `roach@Roach:22`; RemoteForward Roach `127.0.0.1:22052` → JUPI `127.0.0.1:22`	JUPI key `JUPI-to-Roach unattended transport`; Roach host key pinned (`StrictHostKeyChecking yes`)	`jupi-link@roach.service`, Restart always/15 s, ServerAlive 30 s×3. Use from Roach: `ssh -p 22052 pi@127.0.0.1`
3	**FC-Link tunnel**	JUPI `127.0.0.1:18765` → NIKOLAI `<nikolai-tailnet-ip>:8765` (via Roach); `127.0.0.1:18766` → Roach loopback `:8765`	same JUPI→Roach key	`jupi-link@fclink.service`. FC-Link itself is HMAC-authenticated
3′	**FC-Link server**	inbound tcp/8765 on + 127.0.0.1	HMAC; nftables allows LAN only from ,	`fclink.service` (fclink 1.3.3)
4	**NIKOLAI Codex**	NIKOLAI → (via Roach) → JUPI tcp/22	`pi` authorized_keys line 4: `Codex-NIKOLAI-to-JUPI`	Last seen Nov 10 7:54 AM from Roach's address
5	**Gadget lane**	JUPI → Muse cloud, TLS 443 (outbound only)	Muse pairing (sealed, not documented)	`musegadget.service` as **root**, `--run-as pi`; display name JUPI; node `homelink-3364d7`; `system.run` via `/run/musegadget/musegadget.sock` (`srw-rw---- root:pi`)

#	Lane	Direction / transport	Auth	Notes
6	**jupi-agent**	local unix socket, pi/root only	filesystem permissions	durable jobs; `jupi-agent run --peer` executes on Roach as `roach`
7	**Desktop agents**	outbound 443	app logins (sealed)	ChatGPT 26.1103.32239 + `codex app-server`; Grok Bot 0.79.3 (local-exec daemon in `~/.grokbot`)

There is no Tailscale on JUPI (`tailscale` not installed). Tailnet reachability is only through Roach.

8.3 Keys: fingerprints and comments only

Where	Fingerprint	Comment / role
JUPI `~pi/.ssh/authorized_keys` #1	`SHA256:aEMbT86C…Z1Q`	`roach@roach` (ED25519): Roach's operator key
JUPI `~pi/.ssh/authorized_keys` #4	`SHA256:XbrOkDNu…ZKU`	`Codex-NIKOLAI-to-JUPI` (ED25519)
JUPI `~pi/.ssh/jupi_roach_ed25519.pub`	`SHA256:9cfyEz78…Dso`	`JUPI-to-Roach unattended transport` (ED25519). Private half stays on JUPI
Roach host key pinned by JUPI	`SHA256:KveFQm/M…H7A`	in `/etc/jupi/ssh/roach.known_hosts`
JUPI host key ED25519	`SHA256:awkaTxgG…zrM`	`root@playstation`. First seen Nov 5 11:51 PM, unchanged since
JUPI host key ECDSA	`SHA256:op1XSU8w…b+Y`	`root@playstation`
JUPI host key RSA-3072	`SHA256:GRg4YZiy…uz8`	`root@playstation`
JUPI host key MLDSA44-ED25519	`SHA256:Y7jG1Xer…ZD8`	`root@playstation` (post-quantum hybrid)
root `authorized_keys`	—	does not exist; root SSH login is disabled anyway

Lines 2–3 of pi's `authorized_keys` are a blank line and a comment. `~pi/.ssh/authorized_key` (no *s*, 0 bytes) is a harmless fossil of a Nov 6 typo in pi's shell history (`curl … >> ~/.ssh/authorized_key`).

Sealed. Never printed, copied or documented: the HDD unlock key (sealed root-only copy), the JUPI→Roach private key, the Grok Bot local-exec credential ((sealed credential file)), the Muse pairing material, app logins, and the fleet ledger directories.

9. Life timeline

All times PST. **Source key:** D = Roach diagnostics `~/ps4-linux/diagnostics`, R = JUPI receipts/backups (file times read in GMT and converted), J = JUPI journal (holds Nov 9 7:54 AM onward only), G = groku-wire records, H = git history, C = rescue checkpoints.

Thu Nov 5: from a dark console to a desktop

Time	Event	Src
3:46 PM	Sony UFS2 top-level directories created on the internal HDD (console storage initialized)	R
~6:01–6:50 PM	USB stick prepared on real clock (`/root/.ssh`, `initramfs.original`, 2.2 GB `psxitarch-live.tar.gz`)	R
(1970 clock)	`fc.sh` boot, `/ps4hss` typo plus the 4 s device race → **rescue shell**. CP01–CP03 written by hand	C
7:46 PM	Roach diagnosis: Ethernet `NO-CARRIER`, PS4 not reached	D
11:32 PM	First payload send refused (port not listening yet)	D
11:45:46 PM	`linux-1024mb.bin` (329,088 B) accepted by the PS4 payload port	D
11:46–11:47 PM	Ethernet carrier drops and returns; OpenSSH 10.6 answers; host key `awkaTxgG…` first seen	D
11:55 PM	Kam confirms the XFCE desktop on the PS4 screen	D

Fri Nov 6: three roots in one day

Time	Event	Src
12:29 AM	SSH preflight: `psxita` and `root` refused (keys only); `pi` is the working account	D
12:34 AM	Root verified: `loop0` ← `/ps4hdd/home/linux.img` (stock 50 GiB) on UFS2	D
12:41 AM	Autostart + Wi-Fi configured: kernel/initramfs/payload copied to `/data`, payload AutoRun	D
≤ 1:02 PM	USB selector fails on `[ -x /newroot/sbin/init ]`. Manual `switch_root` to USB `sda2` (reported)	C
1:02 PM	**CP04**: first desktop on the USB root; Roach SSH as `pi` passes	R
1:06–1:25 PM	**CP05**: one-byte selector repair (`}` → `]`), 10/10 tests, three rollback copies, restart test	R
~1:21 PM	Boot `b00c61e1`: USB root comes up on its own	R
1:33–2:37 PM	"Computer gates": GPU compat libs (LLVM 13), Firefox 159, official ChatGPT/Codex desktop app	R
2:01 PM	Handoff: Roach seat → NIKOLAI seat	R
5:15 PM	JUPI platform installed on the USB root (units, firewall, sshd hardening, `installation.json`)	R
~8:48 PM	Boot `54f4c94e`: **first boot on the JUPI image** (40 GiB, UUID `70dbf870…`)	R
9:02 PM	Disk-only boot bundle backed up (bzImage, both initramfs, selector `usbroot.after.sh`)	R
9:13 PM	Disk-only selector activated (`initramfs cdeaefdb…` on the HDD copy)	R
9:16 PM	Root grown online **40 → 355 GiB**	R
9:17 PM	USB swap retired. 2 GiB `jupi-zram` takes over	R
9:19–9:53 PM	Toolbox, rootless containers, OpenCL (clang/libclc 13) installed with receipts	R
10:35 PM	Cluster enrollment by Ghostie: FC-Link, xchain, Agape, CastorLTE (first authored row #281)	R

Sat Nov 7: qualification day

Time	Event	Src
5:49–8:53 AM	Observability, science, creative and instrument config, each with a checkpoint pushed to Roach	R
11:42 AM	Native warm-boot (kexec) candidate staged, later **held** for missing early GPU firmware	R
11:56 AM	Supervised userspace soft-reboot: SoftRebootsCount 0 → 1, same kernel, state preserved	R
4:35–4:51 PM	Benchmark suite; commit `6641144` claims 2.86 TFLOPS	H
11:15:16 PM	Kernel reboot → boot **`5c74d073`** (current). Cause not recorded	R
11:49–11:51 PM	`jupi-pipeline-bench` replaces the inflated figure; Ghostie display keeper and no-DPMS installed	R

Sun Nov 8 – Mon Nov 9

Time	Event	Src
Nov 8 7:15–8:02 AM	Host watch, UMA bench, rover quality, mission runner added	R
Nov 9 7:54 AM	Oldest entry still in the journal (rotation at 128 MiB)	J
Nov 9 8:39–9:25 AM	Computing lab + knowledge archive (`jupi-knowledge` timer)	R
Nov 9 9:52 AM	Display keeper reordered: zero timeouts first, `-dpms` last	R
Nov 9 9:32 PM	Commit `38277a9`: "JUPI caught its own lie" (perf gates)	H

Tue Nov 10: the freeze

Time	Event	Src
5:28–5:49 AM	Groku read-only probes 1–3	G
5:53 AM	Backups taken; step 1a: zram1, sysctl 90-groku, OOM drop-ins (daemon-reload times out under pressure)	G
6:15 AM	Step 1b: zram1 actually started (4 GiB swap total)	G
6:25 AM	Step 1c: earlyoom 1.9.0 installed and armed	G J
~6:2x AM	MemAvailable ~190 MB, swap 4024/4095, **GTT 3,615,711,232 / 3,618,242,560**	G
7:02–7:17 AM	37 fresh SSH attempts from Roach fail (timeouts, resets in key exchange); ping OK	G
7:31–7:36 AM	Grok Bot app killed. No relief (107 MB available)	G
7:46 AM	A login gets through	J
7:47:03 AM	`sudo systemctl restart lightdm` (pi, Ghostie's session)	J
7:47:28 / 7:47:46	lightdm stopped ("3.4G memory swap peak") / started	J
7:49:27 AM	Autologin session for pi	G
7:50:01 AM	MemAvailable 4343 MB, swap 183 MB. GTT 31 MiB at 7:50:18	G
7:50:17–7:51:11 AM	Kernel: `amdgpu: GPU reset(20/21) failed`, `asic atom init failed`, DP link training failures (uptime-derived times)	R
7:51:09 AM	HDMI re-handshake: `HANDSHAKE_OK`, 1920×1080@60 primary	G
8:09 AM	Watch: `.xsession-errors` +5.49 MB / 30 s from ChatGPT GPU process	G
8:10 AM	As-built scan `JUPI-ASBUILT-SCAN-20261110T1610Z`	G
8:24 AM	Spot check for this manual: 3111 MB available, GTT 30 MiB, `.xsession-errors` 357.6 MB	—

Boots not listed above (e.g. after the 12:41 AM autostart work) have **UNKNOWN** IDs. The journal that would have named them has rotated.

10. Known open issues

Ordered by operational risk. None of these were changed while writing this manual.

10.1 Runaway `~/.xsession-errors` (ACTIVE)

```
8:08 AM   183,732,736 B
8:09 AM   +5,491,898 B / 30 s   (~183 KB/s, ~15.8 GB/day)   watch window 8:09:13-8:09:43
8:24 AM   357,581,223 B          (~175 KB/s over 16.5 min)
writer    ChatGPT --type=gpu-process (pid 346148), repeating:
          SharedContextState::InitializeGL failure driver error : 33364
          ContextResult::kFatalFailure: Failed to Initialize GL for SharedContextState
          GpuChannel: Failed to create SharedImageStub
```

Memory is otherwise fine (≈3.1 GB available, GTT flat ≈30 MiB). Disk is the slow fuse. The log lands in the root image, so it eats the sparse image's real blocks *and* the UFS backing (333 GiB free). That gives weeks, not hours, but it never stops on its own.

Correlated kernel events (hypothesis): right after the 7:47 AM X restart the kernel logged `amdgpu: GPU reset(20) failed`, `GPU reset(21) failed`, `GPU Recovery Failed`, `asic atom init failed!` and repeated DisplayPort link-training failures (7:50–7:51 AM). A GPU left in a partly recovered state would explain an Electron GPU process that can no longer create a GL context. **UNKNOWN** whether desktop GL works for other clients right now. Fix path: Runbook R8 (stop the writer, truncate). Whether a full GPU recovery needs a kernel reboot, with its cold-boot caveats, is **UNKNOWN**.

10.2 The GTT pin can recur

Nothing on JUPI watches GTT. earlyoom watches RAM and swap only, and it avoids Xorg. With ChatGPT, Codex and Grok Bot all running, expect the climb to repeat over hours to days. **Mitigation:** the gauge (6.3) on a schedule, plus the DESKTOP/COMPUTE discipline (6.4). *A GTT watch timer would be a natural next addition. It is proposed here and not built.*

10.3 No warm way in

Roach has no ControlMaster/ControlPersist entry for JUPI (checked 8:24 AM). The next freeze will hit the same wall as 7:02–7:17 AM unless R0 is done first.

10.4 No local escape hatch

Xorg runs with `-novtswitch`, the kernel has no SysRq, and the ttyS0 UART is not initialized. A wedged desktop at the console has no keyboard-only way out.

10.5 Power loss is not self-healing

`cold_boot_verified: false` in every receipt. Return to Linux needs the console OS plus the homebrew loader. Treat JUPI as **must-not-lose-power** until an unattended path is proven.

10.6 Possible silent fallback to the USB root

The selector waits 5 s. In the rescue era the HDD bridge appeared at 6.65 s. If it is late, rc 1 boots the old USB root quietly. After any reboot, check `findmnt /` (R6).

10.7 Smaller items

Item	Detail
Observer timeouts	jupi-nerves 201, host-watch 196, observe 45, knowledge 16, health 16 timeouts since Nov 9 7:54 AM; `jupi-health.service` failed at scan
HDMI EDID = 0 bytes	panel reports `0mm x 0mm`; mode must be forced after each X restart (R4)
No `IgnorePkg`	`mesa-ps4`, `xf86-video-amdgpu-ps4` not fenced from a blanket `pacman -Syu`
Baseline drift	earlyoom (and zram1, sysctl 90-groku) differ from the `jupi-profile` pinned baseline
Stale comment	`90-groku.conf` says "USB loop root". The root is the HDD loop
Unreclaimable slab	1.15 GiB SUnreclaim at scan, cause **UNKNOWN**
Kernel source this boot	USB `PS4BOOT` copy vs HDD copy: **UNKNOWN**
`systemd-coredump@0-4100-327613-0`	failed unit left over from a core dump. Which process crashed is not in the scan (**UNKNOWN**)

11. Receipts appendix

11.1 Source documents for this manual

Document	Location	Size	sha256
As-built scan (primary)	JUPI ~/groku-wire/, NIKOLAI ~/groku-relay/inbox/, Roach ~/groku2/jupi-map/	279,105 B	3559b9342a67edecf5f9adcfbc196f875b53b1e989f1552fab9eb7e496053c8d (verified on Roach and NIKOLAI copies, Nov 10 8:23 AM)
Per-layer working files L1–L6, usbroot.after.sh, irfs/current.cpio.gz	Roach ~/groku2/jupi-map/	—	initramfs cdeaefdb…7d11e21 (matches live)
X-restart verification	JUPI ~/groku-wire/RESULTS-xrestart-20261110T155109Z.md	3,066 B	85c79208fd1cf8a340b35b65127f7fa20ecde67aafcf8c0e652df05b83b5b755
Freeze recovery attempt log	NIKOLAI ~/groku-relay/inbox/RECOVERY-ATTEMPT-20261110T151745Z.md	—	—
Change log (hash-chained)	JUPI ~/groku-wire/CHANGES.md	2,008 B	chain heads f4efa078…, a058f987…
Origin diagnostics	Roach ~/ps4-linux/diagnostics/	—	payload receipt PAYLOAD-SEND-20261105T234546-0800-ROACH2.json

11.2 Boot-critical files

File	Bytes	sha256
/ps4hdd/data/linux/boot/bzImage (= USB copy)	15,692,800	f0008d466329d07d456fda63fe03129f7af0403a4040d357d3a62658e3d17293
/ps4hdd/data/linux/boot/initramfs.cpio.gz (active)	6,778,642	cdeaefdb30c7b60ad25fa478ef41eb365fb5132795257011f753578537d11e21
…/initramfs.before-disk-only-20261107T051314Z.cpio.gz	6,795,645	7eec9f0e7ac9c23b4982e7f3f6c08abcc40142b9b53068e601ddc815d7df39a2
usbroot.sh (root selector, in initramfs)	9,685	f65941df0ea77cbfc542105e5b1f3f4ffecd26ce16be4fdcc9aa8eb9634ac797
init (better-initramfs, in initramfs)	4,003	06cb53715d3beaf97abad7c88a2cf6902cd110dffdda258c430484a1420da8af
/usr/local/bin/jupi (selector-pinned)	11,944	3f5d4eee5787f8991f9a9447e7cfc2f7e1438b77d3b885e2f98e4bdceca12bb1
linux-1024mb.bin (payload)	329,088	e11da9d2f772bd2619587d1bc3bd49b257876d1b298fd4f01e1a1139414b6646
/usr/local/lib/jupi-boot/kexec	—	fa5d442d88e24cbf02f322f1edb09130990aac9c9842c3abf9c0701508733254
/usr/local/sbin/jupi-zram	—	8d9d1f4f499a219aeb74efc5c1908df54900234f232a6696ccab2d5830a440f2
/usr/local/sbin/groku-zram	—	e6d2ca3fd6f62176ea17367da3a03e51397ce094f56af69d0e5444bc5b6c52bd
Root image jupi-linux-2a79fafeb62d61ff.img	381,178,347,520	not hashed (live, 355 GiB)

11.3 Rescue-shell and repair checkpoints

Checkpoint	Item	sha256
CP01	system.txt	9e511a79921b573ea0f850e79eff35e577b295aeb2b42a0bb5121cc966ed5a68
CP01	boot-hashes.txt	a4c440e0240eaaecc5bfbff4193bfb95fbec5fb018efe79f798c694e0d7171a1
CP01	kernel-log.txt	c37d21405cee5726b2564b4cfd6b1b8600877b2b4812c46d96bbb3ac028183ed
CP02	fc.sh (the /ps4hss script)	4c4f902b33743f4441124bf2c4033855f9727fe6630abbf8f42fd21c2fe6143b
CP02	init	5fb35223732bded5653ae72e2a0cec8f8b6d68a3dfd4c805d07da1a354d24de4
CP02	functions.sh	807f36708317373bd2a206b33df822b07690befb5a446b83b68035baa3358b4d
CP03	initramfs.pre-v3.cpio.gz	3a77c2a183d1f8bbd9a3b36fecb30d737020645922260db4a110e22bf95170bf
CP05	source archive (selector with })	8888da8f165bdb1a81612031166bc890cbd6490395cd223ba5691230e9e63d91
CP05	repaired archive (6,775,692 B)	fa7d6f7763ffb5f9f74bf1b32154bd4a961e94b3a9e98e26b53dc187b88fc4d9
CP05	repaired usbroot.sh	c3baf9caf3fadf0e49fb40589048a3c49338eadc09df671996d127d2e8ded5d8

11.4 Nov 10 change receipts

Item	sha256
rollback-step1.sh	372417fcd4bdc09fbfb76d9b98c577ebc8add9348665cded87479c36ffef2303
earlyoom-1.9.0-1-x86_64.pkg.tar.zst	f54e18d52be814ceaed9b2e4829ed158c534f8e6b83900af57817198ef3bba59
recover.sh (staged during the freeze, **never executed**)	bfceda99d6ba000df175e0fea4a33abb4509727e7dfa13c2eec68a4203c76a70
RESULTS-xrestart-20261110T155109Z.md	85c79208fd1cf8a340b35b65127f7fa20ecde67aafcf8c0e652df05b83b5b755

11.5 Boot list

Boot ID	Root	Came up (PST)	Evidence
7e1177e5-de33-4a4d-a2f4-fe99367a1fe8	loop0 ← linux.img (50 GiB stock)	Nov 5 ~11:47 PM	autostart report (initial boot ID)
a6a81461-e092-489a-923b-db43b6e695e1	/dev/sda2 USB	before Nov 6 1:06 PM	CP05 report
b00c61e1-6e1e-4d26-ab31-0caf281bcd82	/dev/sda2 USB (111.5 GB fs)	Nov 6 ~1:21 PM	boot receipt (uptime 6405 s at 3:08 PM)
54f4c94e-2a9d-443a-8c03-321c3ae6a714	loop0 ← JUPI image, 40 GiB	Nov 6 ~8:48 PM	boot receipt (written 8:50 PM)
soft-return f3be767b-…	same kernel boot, userspace restart	Nov 7 ~11:56 AM	SOFT-RETURN-*.json
5c74d073-6f75-42c5-b5d4-566cc6090faa	loop0 ← JUPI image, 355 GiB	**Nov 7 11:15:16 PM** (btime 1794122116)	boot receipt, journal

11.6 Identity anchors

Item	Value
Root FS UUID	70dbf870-bcb6-4be9-9fb5-f25c4718b71a (label JUPIROOT)
USB boot partition	891E-578E (PS4BOOT)

Item	Value
USB fallback root	`2ec1035f-9275-4d5b-bd46-c6e40122ea8d` (`psxitarch`)
HDD GPT disk GUID	`66FB93BD-D972-11F1-8753-709E29577E23`
Endpoint ID	`jupi-f04a6e42c85374a67f331992` (`/etc/jupi/identity.json`)
Muse node	`homelink-3364d7`, display name `JUPI`
Castor anchor	`07b1bb90149ff0e24e4dbf5902dcbfdbf3d81f89132cd572cecd25a7fb2b56ce`

Key fingerprints: Section 8.3.

— end of manual —